

Rúbrica Evaluación: Sumativa N°2

Programación Orientada a Objeto Seguro

Unidades de aprendizaje 2 y 3: Software orientado a objetos, persistencia y seguridad en Python

(ponderación de evaluación 35 %)

Área Académica	Tecnologías de Información y Ciberseguridad		Carrera	Ingeniería Informática
Sede	Puente Alto		Código	TI3V21
Docente	Michael Arjel	Fecha	Miércoles 7 de octubre de 2026	
Sección	114-2A-F2	Duración	Desde el 28 de septiembre al 7 de octubre	

Nombre Equipo:			
Negocio asignado:			
Integrante 1:			
	Apellido Paterno	Apellido Materno	Nombres
Integrante 2:			
	Apellido Paterno	Apellido Materno	Nombres
Rut integrantes:			

Puntaje Máximo	42	Nota:		Firma Conforme	
Puntaje Obtenido					
Solicita Re-Corrección	Sí	No	Motivo:		

INSTRUCCIONES GENERALES:

- La nota 4.0 se obtiene logrando un 60% del puntaje total, es decir 25 de 42 puntos.
- La evaluación es presencial: una demostración en vivo del software en la clase del miércoles 7 de octubre.
- Antes del miércoles 7 de octubre a las 19:50, el equipo sube al Ambiente de Aprendizaje INACAP el enlace a su repositorio GitHub. El repositorio debe ser público: uno privado o un enlace que no abre se considera no entregado.
- El programa se ejecuta en vivo desde el repositorio. Lo que no funcione en la demostración se evalúa como no funcionando.
- La nota es la misma para todo el equipo. La defensa individual se realiza en la Evaluación Sumativa N°3.
- Puede usar asistentes de inteligencia artificial, pero debe documentar en el README qué adoptó, modificó o descartó, y por qué.

Aprendizajes esperados:

2.1.- Construye una solución de software en Python para resolver una problemática planteada, considerando el paradigma de programación orientada a objetos, la conexión segura a bases de datos y la aplicación de criterios técnicos para validar e integrar recomendaciones generadas por herramientas de IA.

3.1.- Desarrolla scripts en Python para consumir servicios externos y persistir datos localmente, aplicando mecanismos de seguridad de la información y considerando herramientas de IA como apoyo al desarrollo.

Criterios de evaluación:

2.1.2.- Aplica principios de programación orientada a objetos en la solución desarrollada en Python, con coherencia con el modelo diseñado y sin automatización del código.

2.1.3.- Emplea librerías oficiales de Python para la conexión a bases de datos, según el sistema gestor seleccionado y los requerimientos de la solución.

2.1.4.- Utiliza mecanismos de manejo de errores y excepciones mediante estructuras de control orientadas a la estabilidad y seguridad del software.

3.1.1.- Programa scripts en Python en el consumo de servicios externos, utilizando librerías oficiales.

3.1.2.- Utiliza mecanismos de seguridad en el acceso y procesamiento de datos mediante técnicas de autenticación y validación de entradas en los scripts desarrollados.

3.1.3.- Aplica mecanismos de manejo de errores en el consumo de servicios externos, mediante estructuras de control que permitan la continuidad y estabilidad del sistema.

3.1.4.- Ajusta la seguridad del código desarrollado con apoyo de herramientas de IA, documentando y justificando las decisiones técnicas adoptadas.

Presentación de la actividad

Durante las unidades 2 y 3 construimos en conjunto el sistema del taller mecánico: las clases del modelo en Python, la base de datos, la validación de los datos y el consumo de una API. Esta evaluación consiste en demostrar en vivo el sistema de su propio negocio, completo y funcionando.

Es una actividad grupal, con los mismos equipos y el mismo negocio de la Evaluación Sumativa N°1, y corresponde al 35 % de la nota del módulo. Después de la demostración reciben retroalimentación, que deben incorporar para la presentación oral de la Evaluación Sumativa N°3, el martes 13 y miércoles 14 de octubre.

El sistema debe cubrir los cinco requisitos mínimos de la ficha del negocio: tres subtipos con un método que se comporta distinto en cada uno, al menos un dato con validación, una transacción con líneas de detalle, dos reglas que impidan una operación, y un precio que dependa de un indicador externo.

Actividades

Actividad N°1: Construcción del sistema

- **Modelo.** Las clases del diagrama de la Evaluación Sumativa N°1 en la carpeta `model/`, una por archivo. Los subtipos heredan con `super().__init__()` y sobrescriben el método que varía. Atributos privados, con `property` y validación en el setter.
- **Reglas.** Una excepción propia por cada una de las dos reglas que impiden una operación.
- **Persistencia.** Base de datos SQLite, con una carpeta `dao/` separada del modelo. CRUD completo de las entidades principales, la transacción guardada con sus líneas de detalle y todas las consultas parametrizadas. El programa crea las tablas si no existen.
- **Validación.** Las entradas del usuario se validan antes de usarse, y un dato inválido se rechaza con un mensaje sin detener el programa.
- **API.** El precio que depende de un indicador externo obtiene su valor desde una API pública con `requests` (por ejemplo, `mindicador.cl`). La consulta queda en una función o clase propia, con tiempo máximo de espera, y si la API falla el programa lo informa y sigue funcionando.

Actividad N°2: README del repositorio

El archivo `README.md` contiene: integrantes y negocio; cómo instalar y ejecutar el programa; las decisiones de seguridad (inyección SQL y validaciones); y al menos un ejemplo concreto de código sugerido por la IA que adoptaron, modificaron o descartaron, con la razón técnica.

Actividad N°3: Demostración en vivo

Cada equipo dispone de unos 10 minutos. Ejecutan el programa desde el repositorio y muestran, en este orden:

1. Un dato inválido rechazado por la validación.
2. El CRUD de una entidad principal: crear, listar, modificar y eliminar.
3. Una transacción con sus líneas de detalle, y que quedó guardada en la base de datos.
4. Las dos reglas del negocio impidiendo una operación.
5. El precio calculado con el indicador obtenido desde la API.
6. Qué hace el programa cuando la API no responde.

Al terminar, el docente entrega la retroalimentación al equipo. Presentan primero los equipos voluntarios; el resto lo selecciona el docente.

Requerimiento de presentación y entrega

Formato	Demostración en vivo del programa, ejecutado desde el repositorio GitHub público del equipo, con <code>model/</code> , <code>dao/</code> , <code>main.py</code> , <code>README.md</code> y <code>requirements.txt</code> .
Medio de entrega	Enlace al repositorio en el Ambiente de Aprendizaje INACAP (AAI), en la tarea "Evaluación Sumativa N°2". Un enlace por equipo.
Plazo del enlace	Miércoles 7 de octubre de 2026, 19:50 horas.
Demostración	Miércoles 7 de octubre, en la sala de clases. Primero los voluntarios; el resto lo selecciona el docente.
Equipos	Los mismos de la Evaluación Sumativa N°1. La nota es la misma para todo el equipo.

Instrumento de evaluación

Para verificar lo que han aprendido, luego de desarrollar esta evaluación, revise la demostración y el repositorio de sus estudiantes a través de la siguiente rúbrica.

Criterio	Dimensión /Indicadores	Niveles de desempeño.					Observaciones
		Destacado (7 puntos)	Habilitado (5 puntos)	En desarrollo (3 puntos)	No logrado (1 punto)	Puntaje obtenido	
2.1.2 Aplica principios de programación orientada a objetos, con coherencia con el modelo diseñado	Modelo orientado a objetos coherente con el diagrama	Las clases corresponden al diagrama de la Evaluación Sumativa N°1, o a su versión corregida y explicada en la demostración. Los subtipos heredan con <code>super()</code> y sobreescriben el método que varía; los atributos son privados, con <code>property</code> y validación, y el programa opera con objetos del modelo.	Hay una falla puntual en la herencia o el encapsulamiento, o una diferencia con el diagrama que el equipo no explica.	El polimorfismo se reemplaza por <code>if</code> sobre el tipo, o los datos se manejan como diccionarios sueltos, o varias clases se apartan del diagrama.	No hay clases del modelo, o el programa no se ejecuta.		
2.1.3 Emplea librerías oficiales de Python para la conexión a bases de datos	Persistencia con DAO y consultas parametrizadas	CRUD completo de las entidades principales mediante DAO separados del modelo. La transacción se guarda con sus líneas de detalle. Todas las consultas son parametrizadas y las tablas se crean al iniciar.	Una entidad principal queda con operaciones parciales, o la transacción se guarda sin sus líneas de detalle, o hay SQL dentro de una clase del modelo.	Solo algunas operaciones (por ejemplo, insertar y listar), o alguna consulta concatena texto que escribe el usuario.	No hay base de datos: los datos viven solo en memoria.		
2.1.4 Utiliza mecanismos de manejo de errores y excepciones	Reglas del negocio implementadas como excepciones propias	Cada una de las dos reglas tiene su excepción propia, lanzada desde el método que corresponde y capturada con <code>try/except</code> . El programa informa el error y sigue funcionando.	Ambas reglas tienen excepción propia, pero alguna no se captura o se captura con un <code>except</code> genérico.	Solo una regla está implementada, o las reglas se resuelven con <code>print 0</code> <code>return False</code> .	El programa se cae ante un dato inválido o una regla del negocio.		

Criterio	Dimensión /Indicadores	Niveles de desempeño.					Observaciones
		Destacado (7 puntos)	Habilitado (5 puntos)	En desarrollo (3 puntos)	No logrado (1 punto)	Puntaje obtenido	
3.1.2 Utiliza mecanismos de seguridad mediante validación de entradas	Validación de entradas	Todas las entradas del usuario se validan antes de usarse (tipo, formato y rango). El dato que exige la ficha se valida en el setter, y un dato inválido se rechaza con un mensaje sin detener el programa.	Se validan las entradas, salvo alguna que llega sin validar.	Solo se valida el dato que exige la ficha; el resto de las entradas se usa sin validar.	No hay validación: un dato inválido se guarda o hace caer el programa.		
3.1.1 / 3.1.3 Consume servicios externos y maneja sus errores para la continuidad del sistema	Precio con indicador desde una API, y continuidad ante fallas	El precio usa el valor obtenido con <code>requests</code> , desde una función o clase propia con tiempo máximo de espera. Si la API no responde, el programa lo informa y sigue funcionando.	Consume la API y usa el valor en el precio, pero falta el tiempo máximo de espera o el manejo de algún tipo de falla.	El valor solo se muestra y no se usa en el precio, o un <code>except</code> genérico oculta el error sin informar.	El valor está escrito a mano en el código, o el programa se cae si la API falla.		
3.1.4 Ajusta la seguridad del código con apoyo de IA, documentando y justificando las decisiones	README con decisiones de seguridad y uso de IA	El README permite instalar y ejecutar el programa, explica las decisiones de seguridad y da un ejemplo concreto de uso de IA con su razón técnica.	El README está completo, pero las justificaciones son generales, sin ejemplos concretos.	El README solo explica cómo ejecutar el programa.	No hay README.		
Puntaje total							/ 42

Tabla de puntaje y nota

Escala con 60 % de exigencia sobre 42 puntos totales. La nota 4.0 se alcanza con 25 puntos, destacada en la tabla.

Puntaje	Nota	Puntaje	Nota	Puntaje	Nota	Puntaje	Nota
0	1.0	11	2.3	22	3.6	33	5.4
1	1.1	12	2.4	23	3.8	34	5.6
2	1.2	13	2.6	24	3.9	35	5.8
3	1.4	14	2.7	25	4.0	36	5.9
4	1.5	15	2.8	26	4.2	37	6.1
5	1.6	16	2.9	27	4.4	38	6.3
6	1.7	17	3.0	28	4.5	39	6.5
7	1.8	18	3.2	29	4.7	40	6.6
8	2.0	19	3.3	30	4.9	41	6.8
9	2.1	20	3.4	31	5.1	42	7.0
10	2.2	21	3.5	32	5.2		